

Subway Surfers Reinforcement Learning Project Documentation

DeepSeek

February 19, 2026

Abstract

This document provides comprehensive documentation for the Subway Surfers Reinforcement Learning project. It covers the custom Gymnasium environment that interfaces with the original game via screen capture, OCR, and mouse simulation, as well as the data collection pipelines, online training with PPO, offline training with CQL, and robust mechanisms such as auto-restart and anti-stuck detection. All code examples and explanations are included to facilitate reproduction and further development.

1 Introduction

This project aims to apply reinforcement learning (RL) algorithms to the popular endless runner game *Subway Surfers*. Since the game is a closed-source Windows executable, we encapsulate it as a standard `gym.Env` using computer vision techniques and input simulation. The project is structured into three main phases:

1. **Environment Wrapping and Testing** – Implement screen capture, OCR for game state extraction, mouse-based control, automatic restart, and anti-stuck mechanisms.
2. **Online Training** – Use PPO (Proximal Policy Optimization) to train an agent interactively, collecting preliminary data and verifying the environment.
3. **Offline Training** – Leverage offline RL algorithms (CQL) on a pre-collected dataset to learn a policy without further real-game interaction.

2 Dependencies and Setup

The following Python libraries are required:

```
1 pip install gymnasium opencv-python pyautogui mss numpy torch
   torchvision \
2       stable-baselines3 d3rlpy easyocr pynput
```

Additionally, you need to have the Subway Surfers executable (e.g., `Subway_Surfers.exe`) available, and adjust the screen capture regions and button coordinates according to your game window placement.

3 The Environment Class: SubwaySurfersEnv

The environment class inherits from `gym.Env` and handles all interactions with the game.

3.1 Initialization Parameters

```
1 def __init__(self, render_mode=None, game_exe_path=r"...",
2             stuck_threshold=10, enable_stuck_detection=True):
```

- `render_mode`: `human` (no window) or `rgb_array` (returns image).
- `game_exe_path`: Path to the game executable.
- `stuck_threshold`: Number of steps with unchanged mileage to declare a stuck situation (default 10).
- `enable_stuck_detection`: Enable/disable anti-stuck logic.

3.2 Observation and Action Spaces

- **Action space**: `Discrete(5)` – 0: no operation, 1: left, 2: right, 3: up, 4: down.
- **Observation space**: `Box(low=0, high=255, shape=(120,160,3), dtype=np.uint8)` – downscaled RGB image of the game window.

3.3 Screen Capture and OCR

The environment uses `mss` for fast screen capture and `easyocr` to read digits from specific regions (mileage, coins, and the “Score” screen). The `_ocr_digit` method handles the OCR:

```
1 def _ocr_digit(self, region):
2     img = np.array(self.sct.grab(region))
3     gray = cv2.cvtColor(img, cv2.COLOR_BGRA2GRAY)
4     _, binary = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)
5     result = self.ocr_reader.readtext(binary, allowlist='0123456789'
6     )
7     if result:
8         text = result[0][1]
9         digits = ''.join(filter(str.isdigit, text))
10        return int(digits) if digits else 0
11    return 0
```

The `_get_info` method combines OCR results and returns a dictionary with `mileage`, `coin`, and `collision` (game over) flags.

3.4 Reward Function

Reward is designed to encourage progress and penalize death:

- +0.1 per step (survival bonus).
- +0.01 per mileage increase.
- +0.1 per coin collected.
- -10 upon collision (death).

```
1 def _compute_reward(self, info):
2     reward = 0.1
3     mileage_gain = info["mileage"] - self.last_mileage
4     if mileage_gain > 0: reward += mileage_gain * 0.01
5     coin_gain = info["coin"] - self.last_coin
6     if coin_gain > 0: reward += coin_gain * 0.1
7     if info["collision"]: reward -= 10
8     return reward
```

3.5 Action Execution

Actions are executed by simulating mouse swipes using `pyautogui`. Before each swipe, the mouse is moved to the center of the game window to ensure consistent starting points.

```
1 if action != 0:
2     pyautogui.moveTo(self.center_x, self.center_y)
3 if action == 1:
4     pyautogui.dragRel(-self.swipe_offset, 0, duration=self.
5         swipe_duration)
6 elif action == 2:
7     pyautogui.dragRel(self.swipe_offset, 0, duration=self.
8         swipe_duration)
9 elif action == 3:
10    pyautogui.dragRel(0, -self.swipe_offset, duration=self.
11        swipe_duration)
12 elif action == 4:
13    pyautogui.dragRel(0, self.swipe_offset, duration=self.
14        swipe_duration)
```

3.6 Automatic Restart Logic

The environment supports fully automated game launching and restarting.

- **First launch:** If the game process does not exist, it is started via `subprocess.Popen`. After waiting for loading, it automatically clicks the main menu PLAY button (coordinates (1418,1074)) and then the start button (1155,1066) to enter a game.
- **Normal death:** When `terminated=True` due to collision, the next `reset()` will click the death-screen PLAY button (1401,1123) to restart.
- **Stuck-induced termination:** If a forced termination happens because of prolonged stuck (see next section), the process is killed and a fresh instance is launched.

3.7 Anti-Stuck Mechanism

The game sometimes suffers from “stuck” situations (e.g., character frozen between obstacles) where no mileage increase occurs and the agent cannot die naturally. To handle this, the environment includes a robust anti-stuck mechanism:

1. On each step, compare current mileage with `last_mileage`. If unchanged for `stuck_threshold` consecutive steps, enter **stuck mode**.
2. In stuck mode, the action is forced to 0 (no operation) and a `stuck_terminate_counter` is incremented each step.
3. If stuck mode persists beyond `stuck_terminate_threshold` (default `stuck_threshold * 3`), the episode is forcibly terminated (`terminated=True`) and the reason is stored as `"stuck"`.
4. On the next `reset()`, if the termination reason was `"stuck"`, the game process is killed and a fresh instance is launched, ensuring full recovery.

The state is maintained with class attributes:

```
1 self.stuck_counter = 0
2 self.stuck_mode = False
3 self.stuck_terminate_counter = 0
4 self._last_termination_reason = "normal"
```

4 Data Collection

Offline RL requires a fixed dataset. Two scripts are provided to collect trajectories.

4.1 Random Policy Collection

`collect_random.py` uses random actions to interact with the environment, saving transitions at regular intervals. The data format is a list of dictionaries, each containing `obs`, `action`, `reward`, `next_obs`, and `done`.

```
1 def collect_random(total_steps=80000, save_interval=10000):
2     env = SubwaySurfersEnv(render_mode=None)
3     data = []
4     obs, info = env.reset()
5     for step in range(total_steps):
6         action = env.action_space.sample()
7         next_obs, reward, terminated, truncated, info = env.step(
8             action)
9         data.append({"obs": obs, "action": action, "reward": reward,
10                     "next_obs": next_obs, "done": terminated or
11                     truncated})
12     obs = next_obs
13     if (step+1) % save_interval == 0:
14         with open(f"random_data_{step+1}.pkl", "wb") as f:
15             pickle.dump(data, f)
16     env.close()
```

4.2 Human Demonstration Collection

`manual_demo.py` allows a human player to control the game using keyboard arrow keys, providing high-quality trajectories. It uses `pynput` to capture key presses and releases, mapping them to actions. The design is **trigger-based**: each key press triggers exactly one swipe action, mimicking natural play.

```
1 class ManualRecorder:
2     def __init__(self, total_steps=20000):
3         self.env = SubwaySurfersEnv(render_mode="human")
4         self.current_action = 0
5         self.need_action = False
6         # ... set up keyboard listener
7
8     def on_press(self, key):
9         if key == keyboard.Key.left:
10             self.current_action = 1
11             self.need_action = True
12         # ... other directions
```

The recorded data is saved as `manual_data.pkl` with the same format as random data.

4.3 Merging and Conversion

After collection, multiple pickle files can be merged and converted into a `MDPDataSet` (HDF5) for use with `d3rlpy`. A helper script `merge_datasets.py` is provided.

5 Online Training with PPO

To quickly validate the environment and collect preliminary data, online training with PPO (Stable-Baselines3) is implemented. The script sets up a CNN policy and runs for 1 million steps, saving checkpoints every 100k steps.

```
1 from stable_baselines3 import PPO
2 from stable_baselines3.common.callbacks import CheckpointCallback
3
4 env = SubwaySurfersEnv(render_mode=None)
5 model = PPO('CnnPolicy', env, verbose=1, tensorboard_log='./logs/')
6 checkpoint = CheckpointCallback(save_freq=100000, save_path='./
7     models/',
8                                     name_prefix='subway_ppo')
9 model.learn(total_timesteps=1_000_000, callback=checkpoint)
10 model.save('ppo_subway_final')
```

Training progress can be monitored with TensorBoard.

6 Offline Training with CQL

Once a dataset (e.g., `subway_dataset.h5`) is prepared, offline training with CQL (Conservative Q-Learning) is performed using the `d3rlpy` library.

```

1 from d3rlpy.algos import CQL
2 from d3rlpy.dataset import MDPDataset
3 from d3rlpy.models.encoders import NatureCNNEncoderFactory
4
5 dataset = MDPDataset.load('subway_dataset.h5')
6 cql = CQL(actor_encoder_factory=NatureCNNEncoderFactory(),
7          critic_encoder_factory=NatureCNNEncoderFactory(),
8          discrete_action=True, use_gpu=True)
9 cql.fit(dataset, n_steps=200000, tensorboard_dir='./cql_logs')
10 cql.save_model('cql_subway.pt')

```

The trained model can later be evaluated in the real environment.

7 Testing and Evaluation

A set of test scripts ensures each component works as expected.

7.1 Testing Mouse Swipes

`test_swipe.py` moves the mouse to the game center and performs left/right/up/down swipes sequentially, allowing the user to verify that the game responds correctly.

7.2 Testing Environment Stability

`test_env.py` runs a random agent for a few hundred steps, printing info and rewards. It checks that auto-restart works after death and that OCR returns plausible values.

7.3 Testing Anti-Stuck

You can artificially induce a stuck condition (e.g., by pausing the game or blocking input) to observe whether the environment enters stuck mode and eventually forces a restart.

7.4 Evaluating a Trained Model

`evaluate.py` loads a saved model (either PPO or CQL) and runs it in the environment for several episodes, reporting total reward and steps. The `render_mode` can be set to "human" to watch the agent play.

```

1 def evaluate(model_path, n_episodes=5):
2     model = CQL() # or load with d3rlpy
3     model.build_with_env(SubwaySurfersEnv())
4     model.load_model(model_path)
5     env = SubwaySurfersEnv(render_mode="human")
6     for ep in range(n_episodes):
7         obs, _ = env.reset()
8         total_reward = 0
9         done = False
10        while not done:
11            action = model.predict(obs.reshape(1,-1))[0]

```

```
12         obs, reward, terminated, truncated, _ = env.step(action)
13         total_reward += reward
14         done = terminated or truncated
15         print(f"Episode {ep+1}: total reward {total_reward:.2f}")
```

8 Future Work

- Optimize OCR speed (e.g., use caching, lighter models) to reduce per-step latency.
- Implement parallel environments (e.g., `SubprocVecEnv`) to accelerate online training and data collection.
- Experiment with other offline algorithms (IQL, TD3+BC) and fine-tune hyperparameters.
- Transfer the trained policy to a simplified simulator (e.g., Pygame-based) for faster iteration.